Intelligent and Communicating Systems (ICS)

LAB N°04 – Arduino Communications (PWM – Sensors – Actuators – Raspberry Pi Intro)

Full Name: Amel Sadoun – Group 2SQ2

A. THEORY

1. Pulse Width Modulation (PWM)

Definition: PWM is a technique to simulate an analog output using a digital pin. By switching the pin ON and OFF rapidly, the duty cycle (ratio of ON time to total period) determines the effective output voltage. This is commonly used for controlling LED brightness, motor speed, and audio signals.

2. Comparing Arduino vs Raspberry Pi GPIO, PWM, and Interrupts

2.1 Raspberry Pi: Raspberry Pi is a Linux-based single-board computer. GPIO pins are digital (HIGH/LOW) by default. PWM is achievable via software or hardware timers, and interrupts can be handled via libraries like RPi.GPIO or pigpio.

2.2 Arduino: Arduino boards provide analog inputs (A0–A5) and digital pins capable of PWM. Analog pins use 10-bit ADC (0–1023), PWM pins use 8-bit by default (0–255). Hardware interrupts respond instantly to pin changes.

2.3 Summary Table:

Feature	Arduino	Raspberry Pi
Analog Input	Yes (A0–A5, 10-bit)	No (needs external ADC)
PWM Output	Yes (8-bit, configurable)	Yes (software/hardware)
Interrupts	Hardware (pins D2–D3)	Software/hardware libraries
Operating System	Bare-metal	Linux-based

B. ACTIVITY

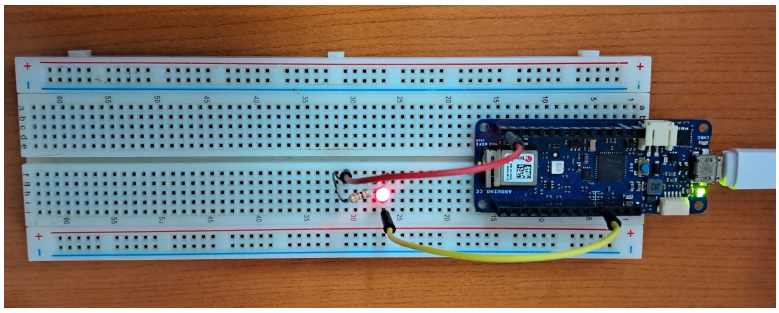
B-1 PWM LED Fading

Objective: Gradually control LED brightness using PWM.

Code:

Listing 1: PWM LED fading

```
const int ledPin = 5;  // Define pin for LED
int brightness = 0;     // Variable to store brightness

void setup() {
  pinMode(ledPin, OUTPUT); // Set LED pin as output
}

void loop() {
  // Increase brightness
  for (int i = 0; i <= 255; i++) {
    brightness = i;
    analogWrite(ledPin, brightness);  // Write PWM value to LED (0-255)
    delay(10);
  }
  // Decrease brightness
  for (int i = 255; i >= 0; i--) {
    brightness = i;
    analogWrite(ledPin, brightness);
    delay(10);
  }
}
```



Explanation: The `analogWrite()` function modulates the LED brightness. The loop increases and decreases the duty cycle smoothly, creating a fading effect.

B-2 LDR Controlling LED via PWM

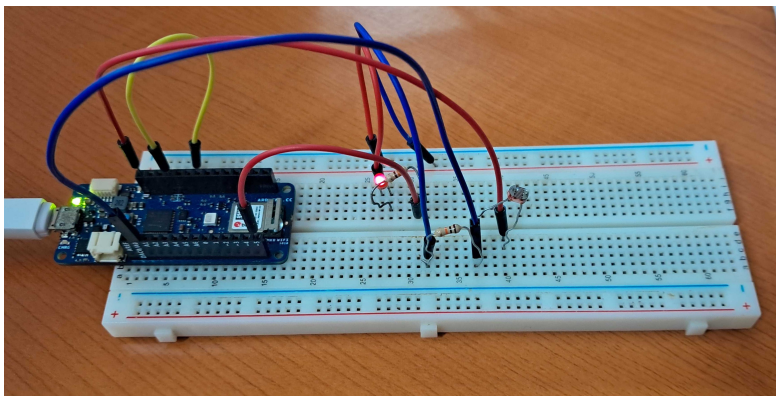
Objective: Adjust LED brightness according to light intensity using an LDR.

Code 1:

Listing 2: LDR controlling LED via PWM

```
const int ldrPin = A1;  // LDR analog input
const int ledPin = 5;   // PWM output to LED

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int ldrValue = analogRead(ldrPin);                  // Read LDR voltage
      (0-1023)
  int brightness = map(ldrValue, 0, 1023, 0, 255);  // Map to PWM range
  analogWrite(ledPin, brightness);                  // Update LED brightness

  Serial.print("LDR Value: "); Serial.print(ldrValue);
```

```
  Serial.print("\tLED Brightness: "); Serial.println(brightness);
  delay(10);
}
```



Explanation: Mapping converts the 10-bit ADC value (0–1023) from the LDR to 8-bit PWM (0–255), ensuring full-range LED control. For example, if `ldrValue = 820`, then:

$$brightness = map(820, 0, 1023, 0, 255) \approx 204$$

Code 2: (this is an additional code of the same circuit that allows us to understand how Arduino PWM handles 10-bit ADC values, showing the binary representation and mapping to 8-bit PWM)

Listing 3: LDR controlling LED with 8-bit cut and mapping

```
const int ldrPin = A0;    // Analog pin connected to the LDR
const int ledPin = 5;     // PWM pin connected to the LED

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int ldrValue = analogRead(ldrPin);            // 10-bit ADC value (0  1023
      )

  // Simulate what analogWrite() actually "sees":
  // it only keeps the lower 8 bits (like taking value % 256)
  byte ledValue = ldrValue & 0xFF;              // Mask to 8 bits (0  255  )

  // Compute what it should have been if properly mapped
  int mappedValue = map(ldrValue, 0, 1023, 0, 255);

  // Send the 8-bit (unmapped) value to the LED
  analogWrite(ledPin, ledValue);

  // Print everything clearly
  Serial.print("LDR (10-bit): ");
  Serial.print(ldrValue);
  Serial.print("\tBinary: ");
  printBinary(ldrValue, 10);

  Serial.print("\tLED actual (8-bit cut): ");
  Serial.print(ledValue);
  Serial.print("\tBinary: ");
```

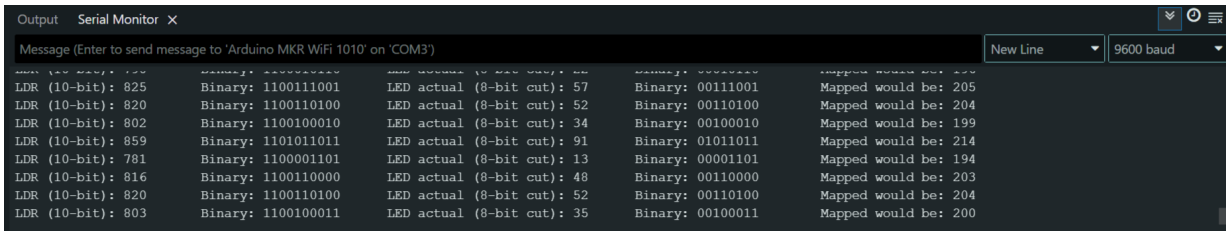
```
  printBinary(ledValue, 8);

  Serial.print("\tMapped would be: ");
  Serial.println(mappedValue);

  delay(500);
}

// Helper: print binary representation with fixed bit width
void printBinary(unsigned int value, byte bits) {
  for (int i = bits - 1; i >= 0; i--) {
    Serial.print((value >> i) & 1);
  }
  Serial.print(" ");
}
```

Explanation: - The LDR gives a 10-bit value (0–1023), but `analogWrite()` only works with 8-bit PWM (0–255). If we send the 10-bit value directly, Arduino keeps only the lower 8 bits (`ldrValue & 0xFF`), which gives an incorrect brightness. That's why we use `map()` to convert the 10-bit value into the correct 8-bit range so the LED brightness matches the LDR reading.

Example from Serial Monitor:

- LDR (10-bit) = 825, Binary = 1100111001
- LED actual (8-bit cut) = 57, Binary = 00111001
- Mapped value (correct 8-bit) = 205, Binary = 11001101

Observation: - Sending raw 10-bit ADC readings without mapping can produce incorrect LED brightness due to truncation. - Mapping ensures the LED fully reflects the sensor range. - The serial monitor outputs with binary columns help visualize exactly how PWM truncates and how mapping corrects it.



B-3 PWM and DAC: Binary Representation

Objective: Compare 8-bit PWM and 10-bit DAC outputs, and observe binary representation of sensor and output values.

Code:

Listing 4: PWM and DAC LED control with binary output

```
const int ldrPin = A0;
const int ledPwmPin = 5;
const int ledAnalogPin = A1;

void setup() {
  pinMode(ledPwmPin, OUTPUT);
  analogReadResolution(10);    // 10-bit ADC
  analogWriteResolution(10);   // 10-bit DAC (if available)
```

```
  Serial.begin(9600);
}

void loop() {
  int ldrValue = analogRead(ldrPin);                    // 0-1023
  int pwmValue = map(ldrValue, 0, 1023, 0, 255);        // 8-bit PWM
  analogWrite(ledPwmPin, pwmValue);
  analogWrite(ledAnalogPin, ldrValue);                 // 10-bit DAC

  Serial.print("LDR: "); Serial.print(ldrValue);
  Serial.print("\tPWM LED (8-bit): "); Serial.print(pwmValue);
  Serial.print("\tDAC LED (10-bit): "); Serial.println(ldrValue);

  delay(50);
}
```

```
Output    Serial Monitor ×
Message (Enter to send message to 'Arduino MKR WiFi 1010' on 'COM3')
LDR: 654        PWM LED (8-bit): 163    DAC LED (10-bit): 654
LDR: 613        PWM LED (8-bit): 152    DAC LED (10-bit): 613
LDR: 612        PWM LED (8-bit): 152    DAC LED (10-bit): 612
LDR: 634        PWM LED (8-bit): 158    DAC LED (10-bit): 634
LDR: 661        PWM LED (8-bit): 164    DAC LED (10-bit): 661
LDR: 648        PWM LED (8-bit): 161    DAC LED (10-bit): 648
LDR: 620        PWM LED (8-bit): 154    DAC LED (10-bit): 620
LDR: 620        PWM LED (8-bit): 154    DAC LED (10-bit): 620
```

Explanation: - The serial monitor shows decimal and binary representations of the 10-bit LDR readings and corresponding 8-bit PWM values. - Binary columns illustrate how values are truncated for PWM. For example:

- LDR (10-bit) = 825, Binary = 1100111001
- LED actual (8-bit) = 57, Binary = 00111001
- Mapped PWM = 205, Binary = 11001101

B-4 Observations

- PWM maps sensor readings from 0–1023 to 0–255, requiring conversion for full-range control.
- DAC (true analog output) can use full 10-bit resolution if supported, providing finer granularity.
- Mapping and binary visualization are essential for debugging and understanding sensor-to-actuator translation.

B-5 Applications of PWM

- Motor speed control in robotics
- LED dimming in lighting systems
- Generating audio tones or alarms
- Simulating analog signals for testing sensors or actuators

C. CONCLUSION

In this lab, we learned:

- How PWM simulates analog output on digital pins.
- Differences between Arduino and Raspberry Pi GPIO, PWM, and interrupt capabilities.
- Mapping techniques to scale 10-bit sensor readings to 8-bit PWM outputs.
- Using DAC for true analog control and understanding the binary representation of sensor and actuator values.
- Practical integration of LEDs and LDRs for responsive embedded applications.